



华南师范大学
SOUTH CHINA NORMAL UNIVERSITY

24级软件工程专业 初级软件实作课

实作报告

实作名称：《BubbleBomb 泡泡大作战》

组长：陈丽文

课题组成员：张晴

完成日期：2026.7

华南师范大学软件学院

2026.7

目录

1、实作项目简介	4
2、 前期准备	4
3、 项目设计	4
3.1 项目开发环境	4
3.2 总体架构设计（系统结构图）	5
3.3 软件包设计	6
3.3.1 软件包总体结构	6
3.3.2 各个软件包职责设计	6
3.3.3 软件包调用关系	8
3.4 类设计（类图）	8
3.2.1.com.bubblebomb.config 包	8
3.2.2.com.bubblebomb.controller 包	9
3.2.3.com.bubblebomb.element 包	10
3.2.4.com.bubblebomb.element.prop 包	16
3.2.5.com.bubblebomb.game 包	17
3.2.6.com.bubblebomb.manager 包	18
3.2.7.com.bubblebomb.show 包	21
4. 系统详细设计与实现	22
4.1 启动模块	23
4.1.1 模块组成	23
4.1.2 启动流程	23
4.1.3 启动模块实现特点	24
4.2 状态管理模块	24
4.2.1 游戏状态设计	24
4.2.2 状态转换过程	25
4.2.3 多回合积分管理	25
4.2.4 暂停实现	26
4.3 玩家控制模块	26
4.3.1 键盘输入处理	26
4.3.2 玩家移动	26
4.3.3 玩家状态	27
4.4 泡泡爆炸模块	27
4.4.1 泡泡放置	27
4.4.2 泡泡倒计时与闪烁	28
4.4.3 水柱扩散	28
4.4.4 连锁爆炸	28
4.4.5 泡泡推动与遥控	28
4.5 道具模块	29
4.5.1 道具生成	29
4.5.2 道具拾取	29
4.5.3 道具功能	29
4.6 地图模块	29

4.6.1 地图类型	30
4.6.2 固定地图加载	30
4.6.3 随机箱子生成	30
4.6.4 特殊地形	31
4.6.5 地图缩圈	31
5. 游戏画面展示	31
5.1 游戏标题与地图选择界面	31
5.2 双人游戏主界面	32
5.3 泡泡放置与爆炸界面	33
5.4 道具掉落与拾取界面	35
5.5 不同地图与特殊地形界面	36
5.6 游戏暂停界面	38
5.7 决胜阶段与地图缩圈界面	39
5.8 决胜阶段与地图缩圈界面	40
6、总结	42
6.1 设计亮点	42
6.1.1 采用模块化的软件包设计	42
6.1.2 支持本地双人实时对战	43
6.1.3 实现了较完整的泡泡爆炸规则	43
6.1.4 设计了多种功能道具	43
6.1.5 支持多地图和随机地图	43
6.1.6 增加了特殊地形效果	44
6.1.7 实现了多回合积分赛	44
6.1.8 设计了决胜倒计时和缩圈机制	44
6.1.9 实现了完整的游戏状态切换	44
6.1.10 采用配置文件管理游戏参数	45
6.2 项目不足	45
6.2.1 游戏平衡性仍需继续调整	45
6.2.2 当前只支持本地双人模式	45
6.2.3 地图和游戏模式数量有限	45
6.2.4 画面和动画效果仍可完善	45
6.2.5 音效和背景音乐不够完善	46
6.2.6 部分核心逻辑集中在 GamePanel 中	46
6.2.7 缺少自动化测试	46
6.3 后续展望	46
6.4 总结	46

1、实作项目简介

《BubbleBomb 泡泡大作战》是一款以经典泡泡堂玩法为基础开发的本地双人休闲竞技游戏，游戏类型为动作策略类游戏，操作简单，节奏明快，具有较强的趣味性和对抗性。游戏中，两名玩家需要在由固定墙、可破坏箱子和特殊地形组成的地图中移动，通过放置泡泡产生十字水柱，破坏箱子、获取道具并攻击对手。玩家可以利用增加泡泡数量、扩大水柱范围、遥控引爆、强化水雷、解药、护盾等道具提升能力，也可能受到减速或方向反转等负面效果。游戏提供多种地图、随机箱子、加速区域、泥地区域、泡泡推动、连锁爆炸、回合积分和动态缩圈等机制。当所有箱子被破坏后，游戏将进入限时决胜阶段，地图安全区域逐渐缩小，最终存活或率先获得指定积分的玩家赢得比赛。

项目使用 Java 编程语言和 Swing 图形界面技术，采用 MVC 设计模式，将界面显示、键盘控制、游戏元素、资源加载和业务逻辑进行分层管理，并运用单例模式、状态机、配置驱动和面向对象多态等设计思想，实现了较为完整的双人泡泡竞技游戏流程。

2、前期准备

在实作之前我们进行了一周的初级软件实作学习，充分了解了 MVC 设计模式，在课堂上，我们紧跟老师的脚步，积累了开发经验。组员们积极交流，确立分工，查找素材。

3、项目设计

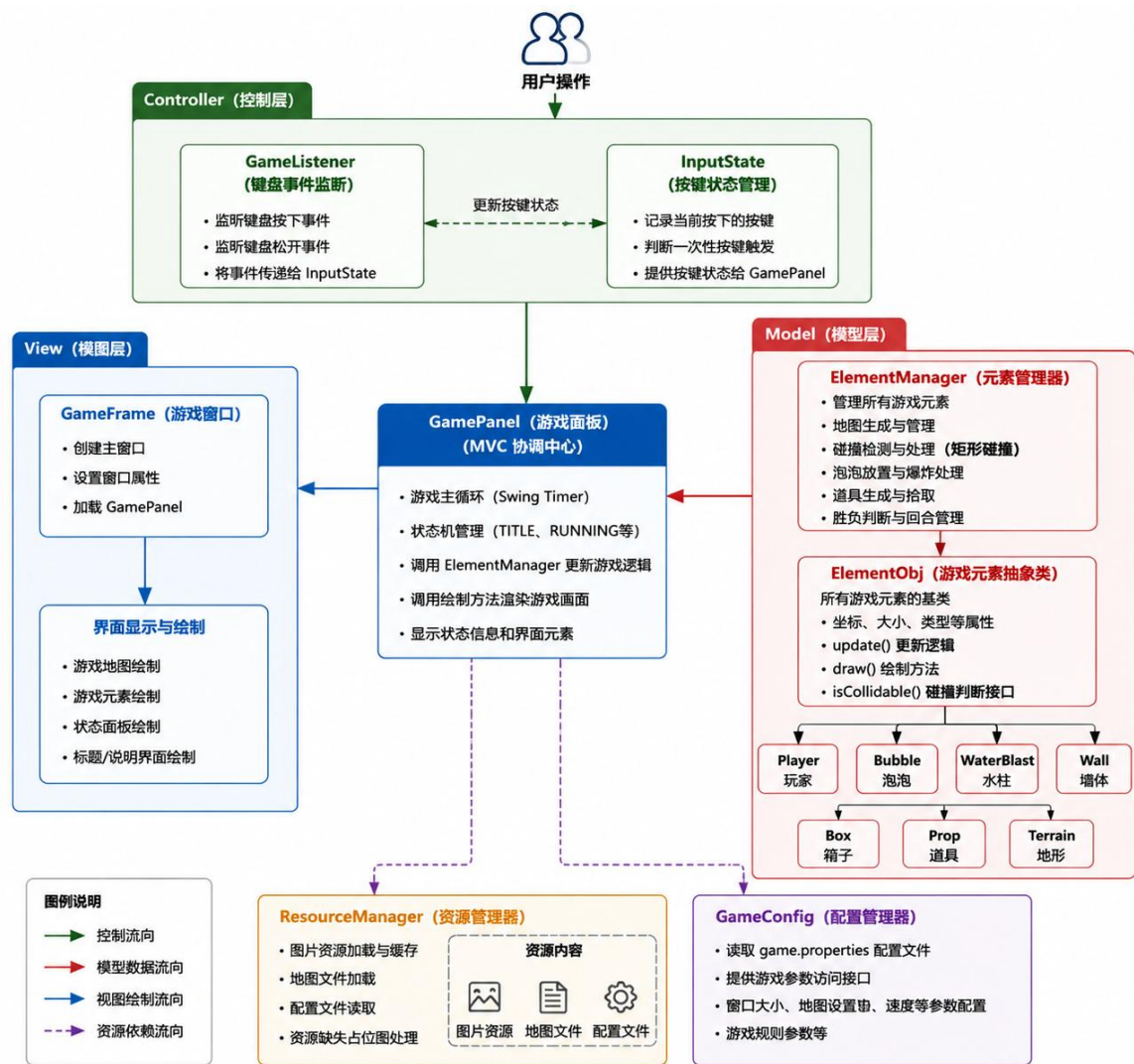
3.1 项目开发环境

项目采用 Java 语言进行开发，基于 Java Swing/AWT 图形界面技术实现游戏窗口、场景绘制以及用户交互功能。项目采用面向对象程序设计思想，并结合 MVC 设计模式进行系统架构设计，提高程序的可维护性和扩展性。项目开发环境如下：

开发过程中利用 Java Swing 完成游戏界面设计，使用 Graphics 绘图技术实现游戏元素渲染，通过面向对象方法对玩家、泡泡、地图、道具等游戏对象进行模块化设计，实现游戏逻辑与界面的分离。

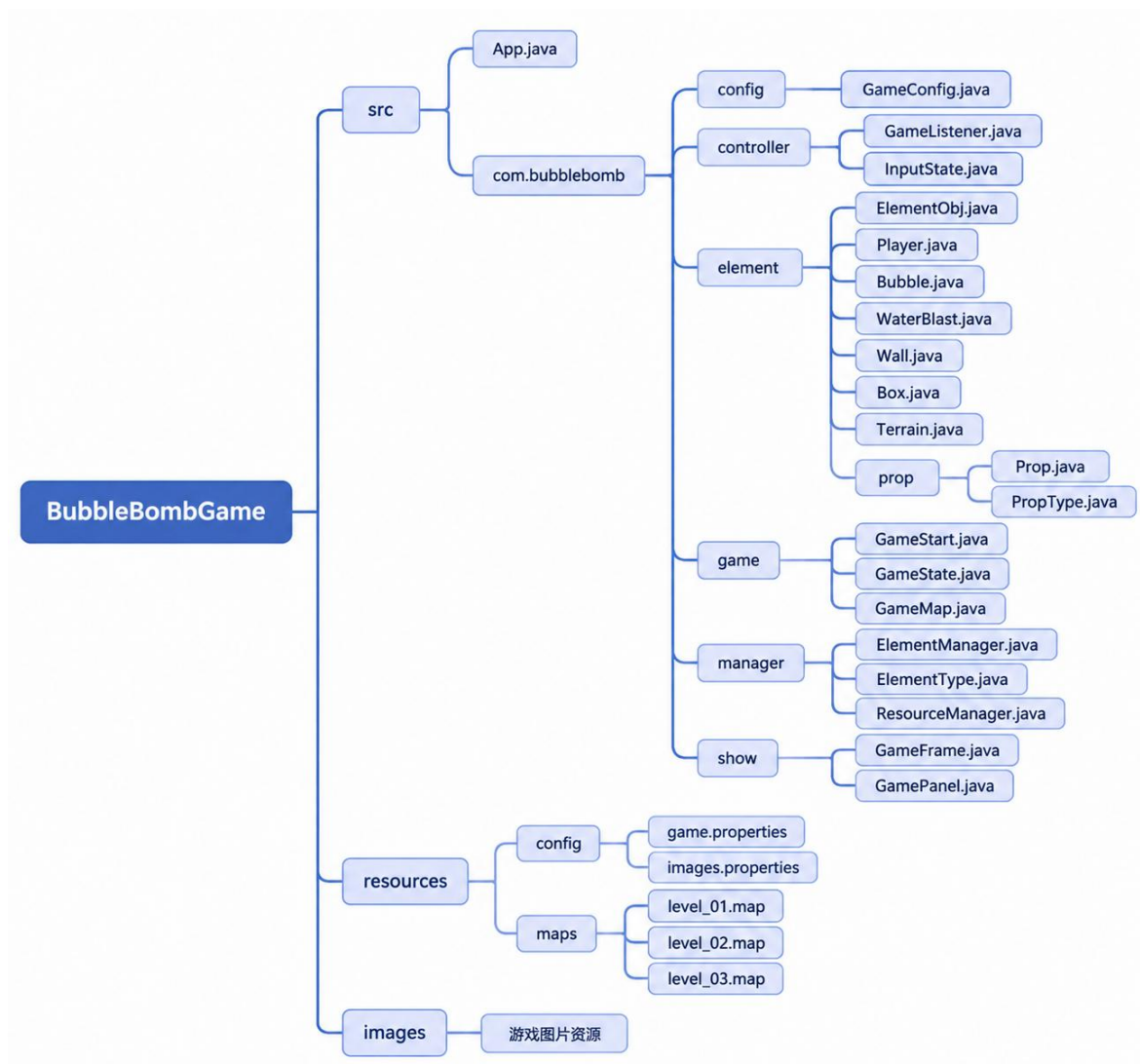
环境类别	配置
操作系统	Windows 10/Windows 11
开发语言	Java
Java 版本	JDK 11 及以上
开发工具	Visual Studio Code
图形界面技术	Java Swing、AWT
项目类型	Java 桌面应用程序
软件架构	MVC 设计模式

3.2 总体架构设计（系统结构图）



3.3 软件包设计

3.3.1 软件包总体结构



3.3.2 各个软件包职责设计

项目按照功能模块划分软件包，各软件包之间具有明确的职责划分。通过模块化设计，将游戏流程控制、界面显示、用户输入、游戏实体管理以及资源配置等功能进行分离，提高了系统的可维护性和扩展性。各软件包职责如下表所示。

软件包名称	主要包含类	主要职责
com.bubblebomb.game	GameStart、GameState、GameMap	负责游戏整体流程控制，包括游戏启动、游戏状态管理以及地图类型定义。 GameStart 负责程序初始化和启动； GameState 负责维护游戏运行状态； GameMap 负责管理游戏地图信息。
com.bubblebomb.show	GameFrame、GamePanel	对应 MVC 中的 View 层，负责游戏界面显示。 GameFrame 负责创建游戏窗口； GamePanel 负责游戏画面绘制、刷新循环以及协调游戏运行过程。
com.bubblebomb.controller	GameListener、InputState	对应 MVC 中的 Controller 层，负责处理用户输入。 GameListener 监听键盘事件； InputState 保存当前按键状态，为玩家移动和技能操作提供输入支持。
com.bubblebomb.element	ElementObj、Player、Bubble、WaterBlast、Wall、Box、Terrain 等	对应 MVC 中的 Model 层，负责定义游戏中的各种实体对象。包括玩家、泡泡、水柱、墙体、箱子以及地形等游戏元素，并实现其属性和行为逻辑。
com.bubblebomb.element.prop	Prop、PropType	负责游戏道具相关功能。 Prop 表示具体道具对象，实现道具生成、显示和拾取逻辑； PropType 定义不同道具类型及对应效果。
com.bubblebomb.manager	ElementManager、ElementType、ResourceManager	负责游戏资源和元素的统一管理。 ElementManager 管理所有游戏对象并处理更新、碰撞、爆炸等核心逻辑； ResourceManager 负责图片、地图等资源加载； ElementType 用于对游戏元素进行分类。
com.bubblebomb.config	GameConfig	负责游戏配置参数管理。读取配置文件中的窗口大小、游戏时间、地图参数等信息，为其他模块提供统一配置访问接口。
resources/config	game.properties、images.properties	负责保存游戏运行所需的配置数据和资源路径信息，实现程序代码与配置数据分离，提高程序灵活性。
resources/maps	level_01.map、level_02.map、level_03.map	负责保存游戏地图文件，包括地图布局、墙体位置以及特殊区域信息，便于后续扩展新的游戏地图。
images	游戏图片资源	存放游戏运行过程中使用的图片资源，包括玩家、泡泡、水柱、地图以及道具等素材。

3.3.3 软件包调用关系

项目中各软件包之间通过明确的调用关系进行协同工作，整体采用分层调用结构。程序启动时，由 App 类调用 GameStart 完成游戏初始化，随后创建 GameFrame 游戏窗口，并加载 GamePanel 作为主要游戏显示区域。GamePanel 作为系统核心协调模块，负责连接控制层与模型层，通过调用 controller 包中的 GameListener 和 InputState 获取玩家输入信息，并将操作传递给游戏逻辑模块。

在游戏运行过程中，GamePanel 调用 manager 包中的 ElementManager 对游戏中的所有元素进行统一管理，包括玩家、泡泡、水柱、墙体、箱子以及道具等对象的创建、更新、碰撞检测和状态维护。ElementManager 进一步调用 element 包和 element.prop 包中的具体游戏实体类，实现各对象自身的移动、绘制、碰撞以及交互逻辑。同时，管理器模块通过调用 ResourceManager 加载游戏图片、地图文件等外部资源，并通过 GameConfig 获取游戏运行参数配置。

通过上述软件包调用设计，系统实现了界面显示、用户输入、游戏逻辑和资源管理之间的有效分离。其中，show 包负责界面展示，controller 包负责用户交互，manager 包负责模块协调管理，element 包负责具体游戏对象实现，各模块之间通过清晰的调用关系降低了系统耦合度，提高了项目的可维护性和扩展能力。

3.4 类设计（类图）

3.2.1.com.bubblebomb.config 包

1) GameConfig 类

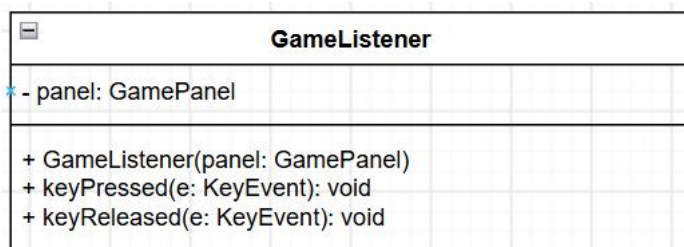
负责游戏参数配置管理，通过读取配置文件获取窗口大小、地图参数、游戏时间以及其他运行参数。该类采用统一配置访问方式，避免程序中出现大量固定参数，提高系统灵活性。



3.2.2.com.bubblebomb.controller 包

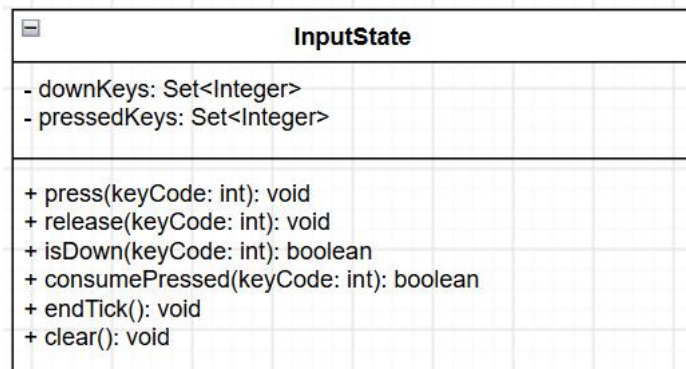
1) GameListener 类

GameListener 类负责监听玩家的键盘输入事件，包括按键按下和释放操作。该类不直接修改游戏对象，而是将输入信息传递给输入状态管理模块，实现输入控制与游戏逻辑的分离。



2) InputState 类

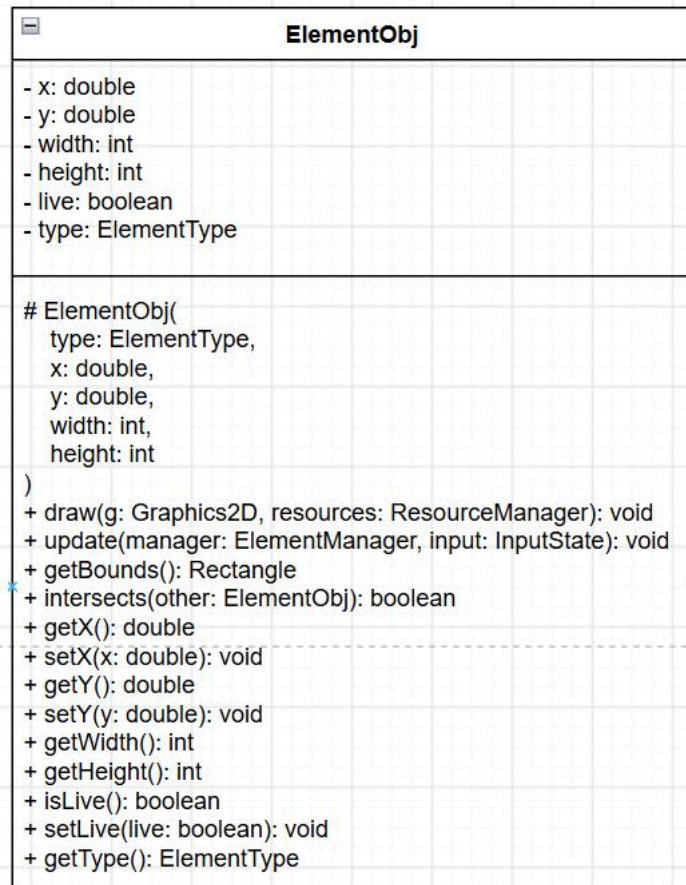
InputState 类用于保存当前玩家输入状态，负责判断按键是否持续按下以及是否触发一次性操作。该类为玩家移动、放置泡泡、暂停游戏等操作提供输入数据支持。



3.2.3.com.bubblebomb.element 包

1) ElementObj 类

ElementObj 类是所有游戏元素的抽象基类，定义了游戏对象共有的属性和行为。该类主要保存元素的位置、大小、生存状态等信息，并提供统一的更新、绘制以及碰撞检测接口。



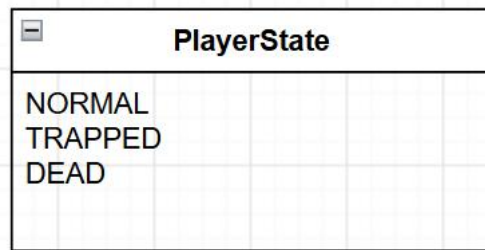
2) Player 类

Player 类用于表示游戏中的玩家角色，负责处理玩家移动、泡泡放置、技能使用以及状态变化等逻辑。同时，该类管理玩家受到攻击、获取道具以及被淘汰等行为。

Player	
- id: int	
- imageKey: String	
- upKey: int	
- downKey: int	
- leftKey: int	
- rightKey: int	
- bombKey: int	
- remoteKey: int	
- direction: Direction	
- state: PlayerState	
- animationTick: int	
- animationFrame: int	
- trappedTicks: int	
- maxBubbles: int = 3	
- activeBubbles: int	
- blastRange: int = 2	
- speed: double = 2.5	
- shield: boolean	
- remoteEnabled: boolean	
- invincibleTicks: int	
- reverseTicks: int	
- slowTicks: int	
- waterMineCharges: int	
- terrainEffect: TerrainType	
- terrainEffectTicks: int	
+ Player(id: int, imageKey: String, x: double, y: double, upKey: int, downKey: int, leftKey: int, rightKey: int, bombKey: int, remoteKey: int)	
+ createPlayerOne(x: double, y: double): Player	
+ createPlayerTwo(x: double, y: double): Player	
+ update(manager: ElementManager, input: InputState): void	
+ updateTerrainEffect(manager: ElementManager): void	
+ draw(g: Graphics2D, resources: ResourceManager): void	
+ getBounds(): Rectangle	
+ getCollisionBounds(x: double, y: double): Rectangle	
+ hit(): void	
+ eliminate(): void	
+ canPlaceBubble(): boolean	
+ bubblePlaced(): void	
+ bubbleReturned(): void	
+ increaseBubbleCapacity(): void	
+ increaseBlastRange(): void	
+ increaseSpeed(): void	
+ grantShield(): void	
+ grantRemote(): void	
+ grantWaterMine(): void	
+ consumeWaterMine(): boolean	
+ applyReverse(): void	
+ applySlow(): void	
+ clearDebuffs(): void	
+ getId(): int	
+ getState(): PlayerState	
+ getBlastRange(): int	
+ getMaxBubbles(): int	
+ getActiveBubbles(): int	
+ getSpeed(): double	
+ hasRemote(): boolean	
+ hasShield(): boolean	
+ getWaterMineCharges(): int	
+ getReverseTicks(): int	
+ getSlowTicks(): int	
+ getTerrainEffect(): TerrainType	
+ getTerrainEffectTicks(): int	

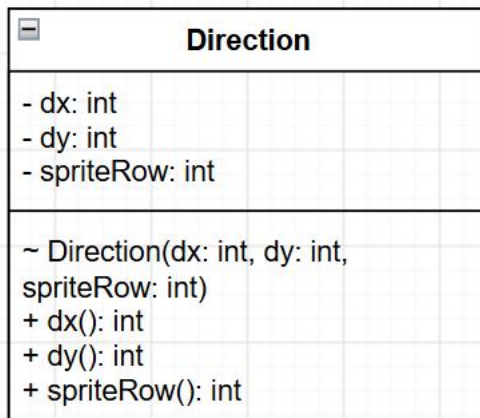
3) PlayerState 类

PlayerState 类用于描述玩家当前状态，通过枚举方式定义玩家正常状态、被困状态以及死亡状态。该类帮助系统根据不同状态执行对应的游戏逻辑。



4) Direction 类

Direction 类用于表示游戏中的方向信息，包括上、下、左、右四个方向。该类主要用于控制玩家移动方向以及泡泡爆炸水柱的扩散方向。



5) Bubble 类

Bubble 类表示玩家放置的泡泡对象，负责管理泡泡的创建、倒计时、爆炸以及连锁反应等功能。同时，该类负责处理泡泡与玩家、墙体以及其他游戏元素之间的交互。

Bubble
- owner: Player - blastRange: int - fuseTicks: int - triggered: boolean - ownerPassing: boolean = true - pushCooldown: int - powered: boolean
+ Bubble(owner: Player, x: int, y: int, size: int) + update(manager: ElementManager, input: InputState): void + draw(g: Graphics2D, resources: ResourceManager): void + blocks(player: Player): boolean + trigger(): void + canBePushed(): boolean + pushTo(x: int, y: int): void + isTriggered(): boolean + getOwner(): Player + getBlastRange(): int

6) WaterBlast 类

WaterBlast 类表示泡泡爆炸后产生的水柱效果，负责控制水柱的生成、显示时间以及与其他游戏对象之间的碰撞处理。当持续时间结束后，水柱会自动从游戏场景中移除。

WaterBlast
- direction: Direction - lifeTicks: int
+ WaterBlast(x: int, y: int, size: int, direction: Direction) + update(manager: ElementManager, input: InputState): void + draw(g: Graphics2D, resources: ResourceManager): void

7) Wall 类

Wall 类表示游戏中的不可破坏墙体，主要负责阻挡玩家移动、限制泡泡传播范围以及阻止水柱继续扩散，是地图环境的重要组成部分。

Wall
+ Wall(x: int, y: int, size: int) + draw(g: Graphics2D, resources: ResourceManager): void

8) Box 类

Box 类表示游戏中的可破坏箱子，负责实现箱子的显示、碰撞检测以及被水柱破坏后的道具掉落逻辑，为游戏增加随机性和可玩性。

Box
+ Box(x: int, y: int, size: int) + draw(g: Graphics2D, resources: ResourceManager): void

9) Terrain 类

Terrain 类用于表示游戏中的特殊地形区域，负责记录地形类型并影响玩家移动效果，例如加速地面和减速泥地等。

Terrain
- terrainType: TerrainType
+ Terrain(x: int, y: int, size: int, terrainType: TerrainType) + draw(g: Graphics2D, resources: ResourceManager): void + getTerrainType(): TerrainType

10) TerrainType 类

TerrainType 类用于定义不同类型的地形，通过枚举方式管理特殊地形效果，为地形逻辑判断提供统一的数据类型。

 TerrainType
- speedMultiplier: double
~ TerrainType(speedMultiplier: double) + speedMultiplier(): double

3.2.4.com.bubblebomb.element.prop 包

1) Prop 类

Prop 类表示游戏中的道具对象，负责道具的生成、显示、拾取检测以及效果应用。当玩家接触道具后，该类会根据道具类型修改玩家对应属性。

 Prop
- propType: PropType
+ Prop(x: int, y: int, size: int, propType: PropType) + applyTo(player: Player): void + draw(g: Graphics2D, resources: ResourceManager): void + getPropType(): PropType

2) PropType 类

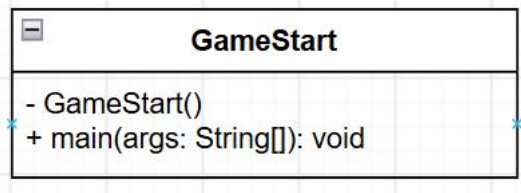
PropType 类用于定义游戏中的各种道具类型，包括增加泡泡数量、扩大爆炸范围、获得护盾以及增强移动能力等效果，为道具系统提供统一管理方式。

 PropType
- imageKey: String - displayName: String
~ PropType(imageKey: String, displayName: String) + imageKey(): String + displayName(): String

3.2.5.com.bubblebomb.game 包

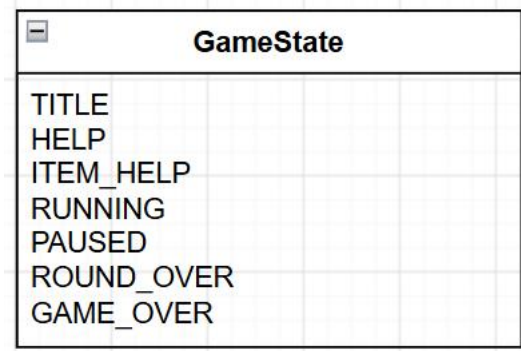
1) GameStart 类

GameStart 类是游戏程序的启动入口，主要负责完成游戏运行环境的初始化，包括加载游戏资源、创建游戏窗口以及启动游戏主界面。该类负责连接程序入口与游戏主体，使整个游戏按照固定流程启动运行。



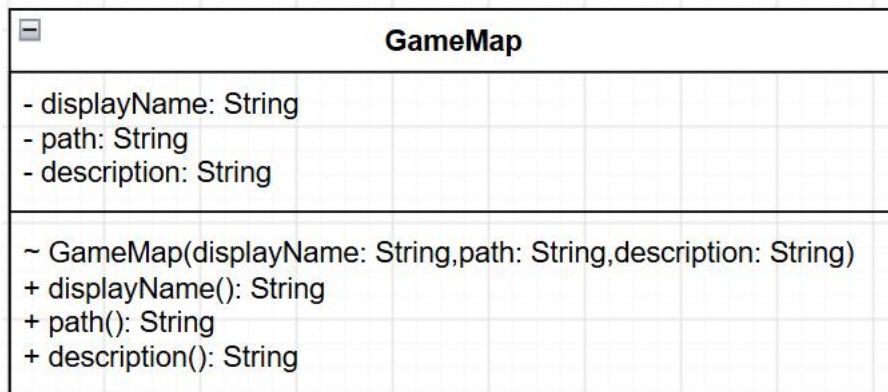
2) GameState 类

GameState 类用于表示游戏当前所处的状态，通过枚举方式管理不同游戏阶段，包括标题界面、帮助界面、游戏运行、暂停、回合结束以及游戏结束等状态。GamePanel 根据当前状态决定游戏逻辑更新方式和界面显示内容。



3) GameMap 类

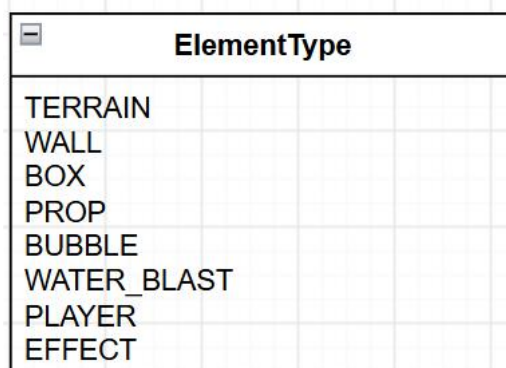
GameMap 类用于定义游戏地图信息，负责保存不同地图的名称、配置文件路径以及地图相关描述。通过该类可以实现固定地图和随机地图的统一管理，方便后续扩展新的游戏场景。



3.2.6.com.bubblebomb.manager 包

1) ElementType 类

ElementType 类用于对游戏元素进行分类管理，包括玩家、泡泡、水柱、墙体、箱子、道具等类型。通过元素类型划分，方便管理器对不同对象进行统一处理。



2) ResourceManager 类

ResourceManager 类负责游戏资源管理，主要用于加载和缓存图片资源、地图文件以及其他配置资源。通过统一资源管理，避免多个模块重复加载，提高程序运行效率。

ResourceManager
- INSTANCE: ResourceManager - imagePaths: Properties - imageCache: Map<String, BufferedImage>
- ResourceManager() + getInstance(): ResourceManager - loadImagePaths(): void + preload(): void + getImage(key: String): BufferedImage - readImage(path: String): BufferedImage + readLines(path: String): List<String> - openStream(path: String): InputStream - createPlaceholder(key: String): BufferedImage

3) ElementManager 类

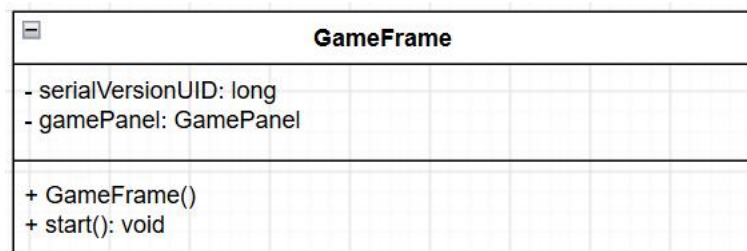
ElementManager 类是游戏元素管理核心类，负责统一管理游戏中的所有对象。该类主要完成元素创建、更新、绘制、碰撞检测、爆炸处理、道具管理以及胜负判断等功能，是连接游戏逻辑和界面显示的重要模块。

ElementManager
- INSTANCE: ElementManager - elements: Map<ElementType, List<ElementObj>> - config: GameConfig - resources: ResourceManager - random: Random - suddenDeathCells: List<Point> - suddenDeathIndex: int - playerOne: Player - playerTwo: Player
- ElementManager() + getInstance(): ElementManager + resetLevel(): void + resetLevel(map: GameMap, boxDensity: int): void + getBaseMap(map: GameMap): List<String> - createDefaultMap(): List<String> - addSymmetricBoxes(lines: List<String>, density: int, spawn1Column: int, spawn1Row: int, spawn2Column: int, spawn2Row: int): void - isOpenAreaConnected(lines: List<String>, boxes: boolean[], rows: int, columns: int, startColumn: int, startRow: int): boolean - isOpenCell(lines: List<String>, boxes: boolean[], row: int, column: int): boolean + update(input: InputState): void - processTriggeredBubbles(): void - processBlastHits(): void - processPropPickup(): void - removeDeadElements(): void - explode(bubble: Bubble): void - dropProp(x: int, y: int): void + canMove(player: Player, newX: double, newY: double): boolean - tryPushBubble(bubble: Bubble, dx: double, dy: double): boolean + getTerrainSpeedMultiplier(player: Player): double + getTerrainType(player: Player): TerrainType + placeBubble(player: Player): void + triggerOldestBubble(owner: Player): boolean + getBoxCount(): int + getElementCount(type: ElementType): int + prepareSuddenDeath(): void + advanceSuddenDeathWall(): boolean - placeSuddenDeathWall(x: int, y: int): void - crushCell(x: int, y: int): void - findAt(type: ElementType, x: int, y: int): ElementObj + draw(g: Graphics2D): void - add(element: ElementObj): void + getPlayers(): List<Player> + getWinnerText(): String + getWinnerId(): int + getTimedResult(): String + getTimedWinnerId(): int getWinnerId(): -1 = 双方仍存活，回合继续 0 = 平局 1 = 玩家一获胜 2 = 玩家二获胜 + clear(): void

3.2.7.com.bubblebomb.show 包

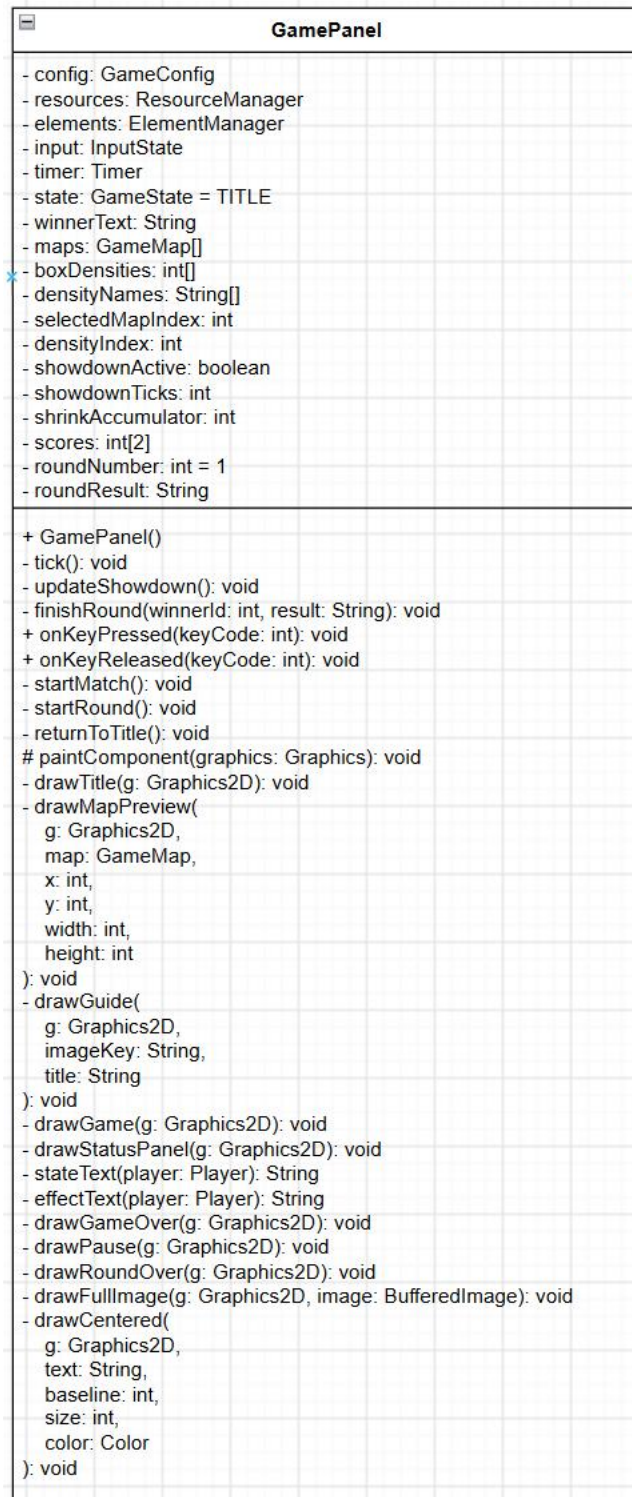
1) GameFrame 类

GameFrame 类负责创建游戏主窗口，是游戏界面的外层容器。该类主要完成窗口大小设置、标题设置、游戏面板添加以及键盘监听注册等功能，为游戏运行提供基础显示环境。



2) GamePanel 类

GamePanel 类是游戏显示和运行控制的核心类，负责游戏画面的绘制、游戏循环控制以及不同游戏状态之间的切换。该类通过调用元素管理器更新玩家、泡泡、地图和道具等对象，并将最新状态绘制到窗口中。



4. 系统详细设计与实现

本系统采用 Java 语言和 Swing 图形界面技术开发，整体按照 MVC 思想划分为显示层、控制层、模型层和管理层。系统通过 **GamePanel** 中的 Swing 定时器实现游戏循环，通过

InputState 保存键盘状态，通过 ElementManager 统一管理玩家、泡泡、墙体、箱子、水柱、道具和特殊地形等游戏元素。

游戏运行时，每次定时器触发都会依次完成输入处理、元素更新、碰撞检测、爆炸处理、胜负判定和画面重绘，从而形成连续的游戏运行效果。

系统主要包含启动模块、状态管理模块、玩家控制模块、泡泡爆炸模块、道具模块和地图模块。

4.1 启动模块

启动模块负责完成游戏运行环境初始化、资源预加载、主窗口创建以及键盘监听器注册等工作。该模块主要涉及 App、GameStart、GameFrame 和 GamePanel 四个类。

4.1.1 模块组成

类名	所在位置	主要职责
App	默认包	提供程序最外层入口
GameStart	com.bubblebomb.game	初始化 Swing 环境并启动游戏
GameFrame	com.bubblebomb.show	创建游戏主窗口
GamePanel	com.bubblebomb.show	创建游戏画面和刷新定时器
ResourceManager	com.bubblebomb.manager	预加载图片和配置资源

4.1.2 启动流程

当用户运行程序时，首先执行 App 类中的 main() 方法。该方法将启动任务交给 GameStart。为了保证 Swing 组件的线程安全，GameStart 使用 Swing 事件调度线程创建游戏窗口。

游戏启动流程：App 启动→调用 GameStart→加载图片与配置资源→创建 GameFrame 主窗口→创建并添加 GamePanel→注册键盘监听器→启动刷新定时器→显示游戏标题界面。

GameFrame 继承 JFrame，负责设置窗口标题、窗口大小、关闭方式和显示位置。窗口内部包含一个 GamePanel 对象，游戏地图、角色、泡泡、道具以及菜单界面都由该面板进行绘制。

GamePanel 继承 JPanel，在构造过程中取得 GameConfig、ResourceManager 和 ElementManager 的单例对象，同时创建 InputState 和 Swing Timer。定时器按照配置文件规定的时间间隔触发游戏刷新。

4.1.3 启动模块实现特点

项目将程序入口与具体游戏逻辑分离。App 只负责启动，GameFrame 只负责窗口管理，而游戏更新和画面绘制集中在 GamePanel 中。这种设计降低了不同功能之间的耦合，便于后续修改窗口尺寸或替换启动方式。

4.2 状态管理模块

状态管理模块负责控制游戏在标题界面、帮助界面、运行状态、暂停状态、回合结束和整场比赛结束等状态之间切换。该模块主要由 GameState 和 GamePanel 实现。

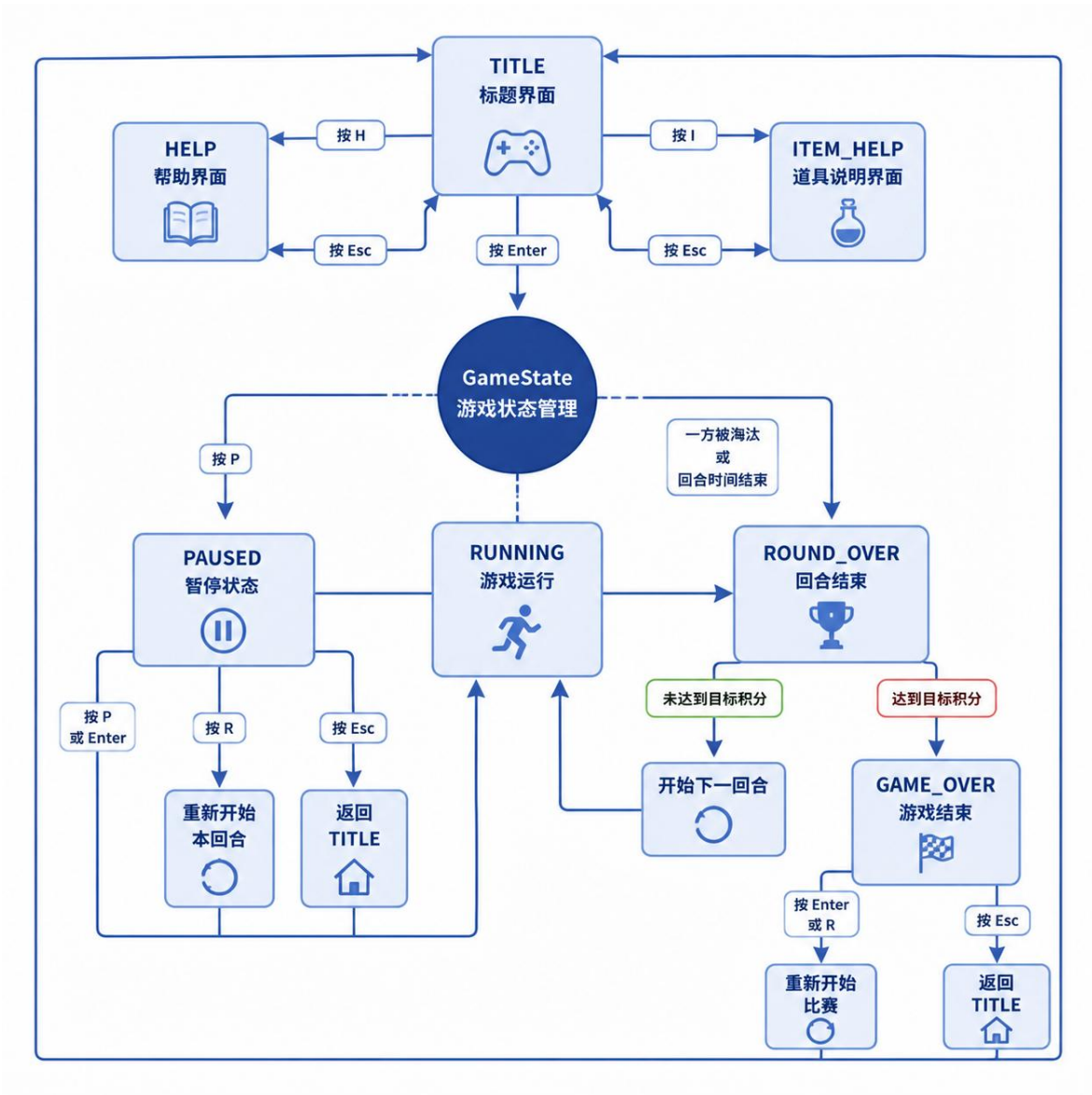
4.2.1 游戏状态设计

项目通过 GameState 枚举定义以下状态：

状态	含义
TITLE	标题和地图选择界面
HELP	游戏操作说明界面
ITEM_HELP	道具说明界面
RUNNING	当前回合正在运行
PAUSED	游戏暂停
ROUND_OVER	当前回合结束
GAME_OVER	多回合积分赛结束

GamePanel 保存当前游戏状态，并根据该状态决定是否更新游戏对象、响应何种按键以及绘制哪个界面。

4.2.2 状态转换过程



4.2.3 多回合积分管理

GamePanel 中保存双方积分、当前回合编号和获胜提示文字。当一名玩家在某个回合中获胜时，其积分增加 1；如果双方同时被淘汰，则判定为平局，双方均不增加积分。

系统默认采用先获得 3 分者赢得整场比赛的规则。目标胜场可以通过 game.properties 中的配置项进行调整。

在所有箱子被破坏后，系统进入两分钟决胜阶段。最后一分钟地图边缘逐渐生成不可破坏墙体，使活动区域不断缩小；最后 30 秒显示闪烁警告。时间结束后，系统根据双方生存状态判定回合结果。

4.2.4 暂停实现

游戏进入 PAUSED 状态后，刷新定时器仍然负责界面重绘，但不再调用游戏对象的更新方法。因此玩家移动、泡泡倒计时、水柱持续时间和特殊状态持续时间都会停止，恢复游戏后再继续计算。

这种处理方式避免了暂停期间泡泡继续爆炸或者玩家异常状态继续计时的问题。

4.3 玩家控制模块

玩家控制模块负责键盘输入采集、玩家移动、方向控制、碰撞检测、泡泡放置以及玩家状态处理，主要涉及 GameListener、InputState、Player、Direction 和 PlayerState。

4.3.1 键盘输入处理

GameListener 继承 KeyAdapter，监听键盘按下和释放事件，并将按键状态传递给 GamePanel。InputState 使用两个集合分别记录：

- 当前持续按下的按键；
- 当前刷新周期中新按下的按键。

持续按键用于角色移动，一次性按键用于放置泡泡、遥控引爆和暂停等操作。这样可以防止一次按键在多帧中被重复执行。两名玩家的默认操作方式如下：

玩家	移动	放置泡泡	遥控引爆
玩家一	W、A、S、D	B	V
玩家二	方向键	Enter	/

4.3.2 玩家移动

Player 类保存角色坐标、移动方向、移动速度、按键配置和当前状态。每次游戏刷新时，系统检查玩家对应的方向按键，并计算新的坐标。

当玩家同时按下两个方向键时，系统对斜向速度进行归一化处理，防止斜向移动比水平或垂直移动更快。

移动前，Player 会通过 ElementManager.canMove() 判断目标位置是否合法，主要检测：

- 是否超过地图边界；
- 是否与固定墙体碰撞；

- 是否与箱子碰撞；
- 是否与阻挡状态的泡泡碰撞；
- 是否与缩圈生成的墙体碰撞。

为了提高狭窄通道中的转向灵敏度，玩家的碰撞矩形小于角色图片显示范围，从而减少角色在墙角被卡住的问题。

4.3.3 玩家状态

PlayerState 定义了三种玩家状态：

- NORMAL：可以正常移动和放置泡泡；
- TRAPPED：被水柱击中后困在水泡中；
- DEAD：已经被淘汰。

玩家被水柱击中时，如果拥有钻石卡护盾，则消耗护盾并抵挡本次伤害；否则进入 TRAPPED 状态。被困倒计时结束后，玩家转为 DEAD 状态。

玩家还可能受到方向反转、减速、泥地减速和加速地形等效果影响。这些状态由 Player 内部的持续时间字段进行管理。

4.4 泡泡爆炸模块

泡泡爆炸模块是游戏的核心规则模块，主要负责泡泡放置、倒计时、闪烁、推动、遥控引爆、水柱生成、连锁爆炸和玩家受击等功能。该模块主要涉及 Bubble、WaterBlast、Player 和 ElementManager。

4.4.1 泡泡放置

玩家按下泡泡键后，GamePanel 检测到一次性按键，并调用 ElementManager.placeBubble()。

放置前需要判断：

- 玩家是否处于正常状态；
- 当前已放泡泡数是否小于泡泡容量；
- 当前格子是否已经存在泡泡；
- 当前格子是否允许放置泡泡。

泡泡放置成功后，玩家的活动泡泡数量增加。泡泡爆炸并移除后，该数量减少，使玩家能够继续放置新的泡泡。玩家初始可以同时放置 3 个泡泡，拾取泡泡道具后可以增加数量。

4.4.2 泡泡倒计时与闪烁

每个 Bubble 对象内部保存剩余倒计时。游戏运行时，每次刷新都会减少该数值。当倒计时进入最后阶段时，泡泡使用不同颜色交替绘制，形成闪烁效果，提醒玩家泡泡即将爆炸。

泡泡满足以下任一条件时发生爆炸：

- 自身倒计时结束；
- 被其他泡泡的水柱击中；
- 拥有遥控能力的玩家主动引爆。

4.4.3 水柱扩散

泡泡爆炸后，以泡泡所在格子为中心，分别向上、下、左、右四个方向生成水柱。水柱传播距离由放置者的爆炸范围决定。

传播规则如下：

- ① 遇到地图边界时停止；
- ② 遇到不可破坏墙体时停止，墙体位置不生成水柱；
- ③ 遇到箱子时破坏箱子，并停止该方向的传播；
- ④ 遇到其他泡泡时触发该泡泡爆炸；
- ⑤ 遇到玩家时执行受伤处理；
- ⑥ 在普通空地上继续向下一格传播。

4.4.4 连锁爆炸

当水柱接触其他泡泡时，系统将该泡泡标记为立即爆炸。ElementManager 会持续处理被触发的泡泡，直到当前帧中不存在新的待爆炸泡泡，从而形成连锁爆炸效果。

过程如下：泡泡 A 爆炸→生成四个方向的水柱→水柱接触泡泡 B→泡泡 B 被立即触发→泡泡 B 继续生成水柱。

4.4.5 泡泡推动与遥控

玩家接触泡泡并继续向该方向移动时，如果泡泡下一格为空，系统会将泡泡推动一格。泡泡推动后设置短暂冷却时间，避免一次移动连续跨越多个格子。

泡泡放置和遥控引爆使用不同按键。遥控器道具生效后，玩家可以主动引爆自己最早放置且仍然存在的泡泡。

4.5 道具模块

道具模块负责道具生成、显示、拾取和效果应用，主要由 `Prop` 和 `PropType` 实现。

4.5.1 道具生成

箱子被水柱破坏后，系统按照设定概率判断是否生成道具。如果需要生成，则从所有道具类型中随机选择一种，并在原箱子位置创建 `Prop` 对象。

未被玩家拾取的道具保留在地图中。当道具被水柱击中或缩圈墙体覆盖时，可以被移除。

4.5.2 道具拾取

在每次游戏刷新中，`ElementManager` 检查玩家碰撞范围是否与道具相交。如果发生相交，则调用 `Prop.applyTo()` 将道具效果应用到玩家，并将该道具从元素集合中删除。

4.5.3 道具功能

道具	实现效果
泡泡道具	增加可同时放置的泡泡数量，上限为 8
蓝色药水	增加水柱传播范围，上限为 8
紫色药水	清除减速和方向反转效果
遥控器	获得主动引爆泡泡的能力
强化水雷	下一枚泡泡倒计时减半，范围增加 2 格
红色恶魔	使拾取者在一定时间内减速
紫色恶魔	使拾取者在一定时间内方向反转
金卡	提升当前回合的基础移动速度
钻石卡	提供一次抵挡水柱伤害的护盾

不同道具通过 `PropType` 进行统一定义，`Prop` 只负责保存类型并调用对应效果。这种设计便于后续增加新道具，只需扩展道具枚举和效果处理逻辑。

4.6 地图模块

地图模块负责固定地图加载、随机地图生成、箱子密度设置、玩家出生点设置、特殊地形处理以及决胜阶段缩圈。该模块主要涉及 `GameMap`、`ElementManager`、`Terrain`、`TerrainType` 和地图配置文件。

4.6.1 地图类型

系统当前提供以下地图：

地图	主要特点
经典对称地图	固定石柱分布，整体路线均衡
开放花园地图	空间较大，包含加速地面
泥地迷宫地图	通道较窄，包含泥地减速区域
随机地图	根据规则随机生成箱子布局

玩家可以在标题界面使用左右方向键选择地图，并使用上下方向键设置箱子密度。界面同时显示地图缩略图和地图特点说明。

4.6.2 固定地图加载

固定地图使用 `.map` 文本文件保存。`ResourceManager` 读取文件内容，`ElementManager` 根据不同字符创建对应元素，例如：

- 固定墙体；
- 玩家一出生点；
- 玩家二出生点；
- 普通道路；
- 加速地面；
- 泥土地面。

使用文本地图配置可以将地图数据与 `Java` 代码分离，增加新地图时只需创建新的地图文件并在 `GameMap` 中注册。

4.6.3 随机箱子生成

系统生成随机箱子时遵循以下规则：

- ① 箱子不能生成在固定墙体上；
- ② 箱子不能覆盖特殊地形；
- ③ 两名玩家出生点附近至少保留两个出口；
- ④ 箱子按照中心对称方式成对生成；
- ⑤ 两名玩家获得相对公平的活动空间；
- ⑥ 每次放置箱子后进行连通性检测；
- ⑦ 不允许箱子将某一片区域完全封死。

连通性检测采用广度优先搜索思想，从可通行位置出发遍历地图。如果增加某个箱子后导致可通行区域被分割，则取消该箱子的生成。

4.6.4 特殊地形

TerrainType 用于定义地形效果：

- 加速地面使玩家移动速度提高；
- 泥地使玩家移动速度降低。

玩家离开特殊地形后，效果仍会持续一段时间。加速效果默认持续 2 秒，泥地减速效果默认持续 1 秒，持续时间可以在配置文件中修改。

4.6.5 地图缩圈

当所有箱子被破坏后，系统进入决胜阶段。在最后一分钟，ElementManager 按照从地图外围到内部的顺序生成不可破坏墙体。

缩圈墙体采用对称方式生成，尽量保证双方公平。当墙体生成时，会清除所在位置的泡泡、道具、水柱和地形；如果玩家处于该位置，则直接判定该玩家被淘汰。随着墙体不断向中心推进，玩家可活动区域逐渐减小，从而避免比赛长时间无法结束。

5. 游戏画面展示

5.1 游戏标题与地图选择界面

游戏启动后进入标题界面。玩家可以选择经典对称、开放花园、泥地迷宫或随机地图，并设置箱子密度。界面右侧显示当前地图的缩略图和特点说明，按下确认键后进入游戏。



图 1.游戏标题与地图选择界面

5.2 双人游戏主界面

游戏主界面由顶部比赛信息区、中间地图区域和右侧玩家状态区组成。顶部显示当前回合、双方积分和剩余箱子数量，地图区域显示玩家及游戏元素，右侧显示泡泡数量、水柱范围和特殊能力。



图 2.双人对战主界面

5.3 泡泡放置与爆炸界面

玩家放置泡泡后，泡泡经过倒计时发生爆炸，并向上下左右四个方向生成水柱。水柱遇到不可破坏墙体时停止，遇到箱子时破坏箱子，遇到其他泡泡时可以触发连锁爆炸。



图 3.泡泡放置与闪烁效果



图 4.水柱扩散与连锁爆炸效果

5.4 道具掉落与拾取界面

箱子被水柱破坏后有一定概率生成道具。玩家接触道具后可以获得泡泡数量提升、水柱范围提升、遥控引爆、速度提升或一次性护盾等能力，部分特殊道具也会对玩家产生减速或方向反转效果。



图 5.道具掉落与拾取界面

5.5 不同地图与特殊地形界面

不同地图拥有不同的固定墙体、箱子分布和特殊地形。玩家进入加速地面后会获得短暂加速效果，进入泥地区域后移动速度降低，离开地形后效果仍会持续一段时间。



图 6.开放花园地图与加速地形



图 7.泥地迷宫地图与减速地形

5.6 游戏暂停界面

游戏运行过程中按下暂停键可以进入暂停状态。暂停期间玩家移动、泡泡倒计时、水柱持续时间和异常状态计时全部停止，玩家可以继续游戏、重新开始本回合或返回标题界面。



图 8.游戏暂停界面

5.7 决胜阶段与地图缩圈界面

所有箱子被破坏后，游戏进入两分钟双人决胜阶段。最后一分钟地图边缘逐渐生成不可破坏墙体，使玩家活动区域不断缩小；最后 30 秒显示警告，从而加快比赛节奏并避免双方长时间僵持。



图 9.决胜阶段与地图缩圈界面

5.8 决胜阶段与地图缩圈界面

一方被淘汰或决胜时间结束后，系统根据双方生存状态判定回合结果，并更新双方积分。当任意玩家达到目标胜场后，整场比赛结束，系统显示最终获胜者，并允许玩家重新开始比赛或返回标题界面。



图 10.单回合结算界面



图 11.多回合积分赛结束界面

6、总结

本项目使用 Java 编程语言和 Swing 图形界面技术，设计并实现了一款本地双人泡泡堂游戏。系统采用 MVC 设计思想，将游戏启动、界面显示、输入控制、游戏元素、资源加载和配置管理等功能进行模块化划分。

项目已经实现双人移动、泡泡放置、倒计时爆炸、水柱传播、连锁爆炸、箱子破坏、道具拾取、特殊地形、多地图选择、随机地图、暂停控制、多回合积分赛以及决胜缩圈等功能，基本形成了一个流程完整、规则清晰且具有一定可玩性的双人对战游戏。

6.1 设计亮点

6.1.1 采用模块化的软件包设计

项目根据不同功能划分为 `game`、`show`、`controller`、`element`、`manager` 和 `config` 等软件包，使各部分具有较为清晰的职责。

其中，显示层负责游戏窗口和画面绘制，控制层负责键盘状态处理，模型层负责描述玩家、泡泡、水柱、墙体、箱子、地形和道具等游戏对象，管理层负责游戏元素和图片资源的统一管理。

所有游戏元素均继承抽象类 `ElementObj`，具有统一的位置、大小、存活状态、碰撞区域、更新和绘制方法。该设计减少了重复代码，并方便后续增加新的游戏元素。

6.1.2 支持本地双人实时对战

项目支持两名玩家在同一台计算机上进行对战。玩家一使用 `W`、`A`、`S`、`D` 控制移动，使用 `B` 放置泡泡，使用 `V` 遥控引爆；玩家二使用方向键控制移动，使用 `Enter` 放置泡泡，使用 `/` 遥控引爆。

系统分别记录持续按下的按键和单次触发的按键，使玩家移动保持连贯，同时防止放置泡泡、暂停和遥控引爆等操作被连续重复执行。玩家还可以进行斜向移动，使角色操作更加灵活。

6.1.3 实现了较完整的泡泡爆炸规则

泡泡放置后会自动进行倒计时，并在即将爆炸时产生闪烁效果。倒计时结束后，泡泡以当前位置为中心，向上、下、左、右四个方向生成水柱。

水柱遇到不可破坏墙体时停止，遇到箱子时将箱子破坏，遇到其他泡泡时会触发连锁爆炸，遇到玩家时会使玩家进入被困状态。玩家还可以推动泡泡，获得遥控器后可以主动引爆自己放置的泡泡，增加了游戏的策略性。

6.1.4 设计了多种功能道具

箱子被破坏后有一定概率掉落道具。当前项目包含泡泡数量提升、水柱范围提升、异常解除、遥控引爆、强化水雷、减速诅咒、方向反转、速度提升和一次性护盾等多种道具。

不同道具可以直接修改玩家属性或为玩家附加临时状态。例如，钻石卡可以抵挡一次水柱伤害，紫色药水可以解除减速和方向反转状态，强化水雷可以使下一枚泡泡缩短倒计时并增加爆炸范围。不同道具的组合使每一回合都具有一定的随机性和变化性。

6.1.5 支持多地图和随机地图

项目提供经典对称、开放花园、泥地迷宫和随机地图等不同地图模式。玩家可以在标题界面选择地图、查看地图缩略图，并调整箱子生成密度。

固定地图通过文本配置文件加载，降低了地图数据和 Java 代码之间的耦合。增加新地图时，只需添加地图文件并注册相应的地图信息。

随机地图在生成箱子时会避开固定墙体和玩家出生位置，并保证出生点周围至少有两个出口。系统还会检测地图区域的连通性，防止箱子将某一区域完全封死。箱子采用对称方式生成，使两名玩家拥有相对公平的活动空间。

6.1.6 增加了特殊地形效果

开放花园地图中设置了加速地面，玩家进入后可以提高移动速度；泥地迷宫中设置了泥地区域，玩家进入后移动速度降低。

玩家离开特殊地形后，效果不会立刻消失，而是持续一定时间。加速和减速效果具有更明显的影响范围，可以使玩家利用地形追击对手或躲避水柱，增强了地图的战术作用。

6.1.7 实现了多回合积分赛

游戏不是通过单个回合直接决定最终胜负，而是采用多回合积分赛机制。每个回合结束后，系统会根据双方生存状态判定获胜者，并更新双方积分。

当某一名玩家达到设定的目标胜场后，整场比赛结束并显示最终结果。如果双方同时被淘汰，则当前回合判定为平局，双方均不获得积分。多回合机制降低了单次失误对最终结果的影响，提高了比赛的公平性。

6.1.8 设计了决胜倒计时和缩圈机制

当地图中的箱子全部被破坏后，游戏进入两分钟双人决胜阶段。在最后一分钟，系统从地图边缘向内部逐渐生成不可破坏墙体，使玩家的活动区域不断缩小。

进入最后 30 秒后，界面会显示醒目的倒计时警告。决胜时间结束时，系统根据双方的生存状态判定回合结果。该机制可以避免双方长时间躲避而导致比赛无法结束，同时增强了游戏后期的紧张感。

6.1.9 实现了完整的游戏状态切换

项目设置了标题、帮助、道具说明、运行、暂停、回合结束和比赛结束等多种游戏状态。不同状态下，系统会响应不同的键盘操作并显示相应界面。

暂停游戏后，玩家移动、泡泡倒计时、水柱持续时间和状态效果计时都会停止。恢复游戏后，各项逻辑继续运行，避免了暂停期间泡泡继续爆炸或玩家被异常淘汰的问题。

6.1.10 采用配置文件管理游戏参数

窗口大小、格子尺寸、刷新闻隔、泡泡倒计时、水柱持续时间、目标胜场、决胜时间和地形效果持续时间等参数均保存在配置文件中。

图片资源的逻辑名称和文件路径也通过独立配置文件进行管理。通过这种方式，可以在不修改主要源代码的情况下调整游戏参数和替换图片资源，提高了项目的可维护性和扩展性。

6.2 项目不足

6.2.1 游戏平衡性仍需继续调整

虽然项目已经设置泡泡数量、水柱范围、移动速度和各种道具效果的上限，但道具掉落概率、异常状态持续时间、特殊地形效果和不同地图的箱子密度仍然需要通过更多实际对战进行调整。

部分道具可能对比赛结果产生较大影响，例如遥控器、钻石卡和强化水雷。后续可以根据多次测试结果分别设置不同道具的掉落概率，使游戏过程更加公平。

6.2.2 当前只支持本地双人模式

当前项目需要两名玩家使用同一个键盘进行操作，尚未实现计算机玩家和网络联机功能。当两名玩家同时按下较多按键时，还可能受到键盘硬件按键冲突的影响。

后续可以增加单人对战人工智能、局域网联机或网络匹配功能，也可以支持游戏手柄，从而扩大游戏的使用范围。

6.2.3 地图和游戏模式数量有限

项目目前提供三张固定地图和一种随机地图，虽然不同地图具有加速、泥地等特点，但特殊地图元素的种类仍然较少。

后续可以增加传送门、单向通道、移动平台、冰面、可开关墙体和定时机关等元素。同时可以增加限时赛、生存赛、道具赛和团队赛等游戏模式。

6.2.4 画面和动画效果仍可完善

当前游戏主要使用二维图片和简单图形完成画面显示，能够表达基本的角色移动、泡泡爆炸和水柱效果，但角色动画、爆炸动画、道具拾取动画和界面过渡效果还比较简单。

后续可以增加更完整的精灵动画、粒子效果、画面震动和渐变切换，使游戏画面更加流畅和生动。

6.2.5 音效和背景音乐不够完善

当前项目主要完成了游戏玩法和画面显示，声音系统仍有较大的扩展空间。游戏过程中缺少与放置泡泡、爆炸、拾取道具、玩家受击和回合结束对应的完整音效反馈。

后续可以增加标题背景音乐、地图背景音乐和不同操作音效，并提供音量调节和静音设置，从而提升游戏的沉浸感。

6.2.6 部分核心逻辑集中在 GamePanel 中

GamePanel 同时承担界面绘制、定时刷新、状态转换、积分管理、决胜计时和输入分发等职责。随着游戏功能不断增加，该类的代码量可能继续扩大，从而增加维护难度。

后续可以进一步建立独立的回合管理器、比赛管理器和界面状态管理器，将比赛流程、计时逻辑与画面显示分离，使系统结构更加清晰。

6.2.7 缺少自动化测试

目前项目主要通过人工运行游戏检查移动、碰撞、爆炸和胜负判定，尚未建立完整的单元测试和自动化测试体系。

后续可以针对地图连通性、泡泡传播范围、连锁爆炸、道具效果、积分计算和状态切换编写测试用例，减少修改功能后出现旧功能失效的情况。

6.3 后续展望

针对上述不足，后续可以从游戏内容、系统架构和交互体验三个方面继续完善。

首先，可以增加更多地图、角色、道具和特殊地形，并根据实际对战数据调整游戏参数。其次，可以增加人工智能玩家和网络通信模块，使游戏同时支持单人模式、本地双人模式和网络对战模式。最后，可以完善动画、音效、按键设置、存档和排行榜等功能。

通过进一步优化，项目可以从一个本地双人对战游戏扩展为具有多种地图、多种模式、智能对手和网络联机能力的完整泡泡堂游戏。

6.4 总结

通过本项目的设计与实现，完成了从需求分析、软件包设计、类结构设计、资源配置到游戏编码和功能测试的完整开发过程。在开发过程中，综合运用了 Java 面向对象编程、继承与多态、集合、枚举、单例模式、配置文件、Swing 绘图、键盘事件监听、定时刷新和碰撞检测等技术。

最终实现的游戏具有完整的启动、选择、对战、暂停、结算和重新开始流程，基本达到了项目预期目标。该项目不仅加深了对 Java 面向对象思想和 Swing 游戏开发方式的理解，也为后续学习人工智能、网络通信和复杂游戏架构奠定了基础。